

Teórico

[TDD] Test driven development $\rightarrow$ Método ^{costoso} para desarrollar, desarrollo conducido por pruebas

Técnicas para implementar la Ux story en el práctico (TP)

Proceso de desarrollo

Requerimientos - Análisis - Diseño - Implementación - ^{Testing} Prueba - Despliegue

Lo más costoso son los horas de trabajo, quiere decir que la tarea más costosa es la de testing.

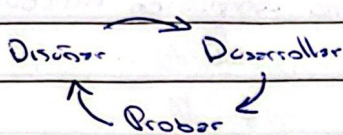
(30% - 50% del costo total del proyecto de SW $\rightarrow$ Prueba) Depende de que tanto nos necesitemos probar, si está en riesgo la vida de personas por ejemplo

Se dio cuenta de que no hace falta tener el software completamente desarrollado para hacer pruebas. De un puzo = no desarrollo para el final, ir adelantando el testing

Caso de prueba: Identificación de situaciones complejas ^{junto con condiciones de inicio} para poder determinar si uno o más criterios de aceptación se cumplen para este SW o no. Se discuten brevemente en los requerimientos. Es el aspecto más importante que sale de la prueba (Testing)

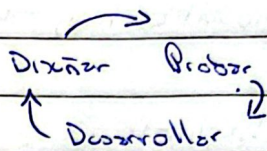
La "confirmación" de la User story tiene que ver con los casos de prueba

Desarrollo tradicional



Los desarrolladores diseñan los
pruebas unitarias después de
escribir código. El asegura-
miento y control de calidad
es más reactivo

TDD (Desarrollo conducido por pruebas)



Primero pensamos como lo vamos a probar y construimos ese componente

Los desarrolladores diseñan los
pruebas unitarias antes de escribir el
código. El aseguramiento y control
de calidad es más proactivo.

- Siempre la actividad Bus de despliegue = los de Testing y despliegue
- Por eso surgió el rol de "DevOps", intermediario entre desarrollo y testing - despliegue.
- Los detalles se transfieren a los pruebas, por eso con más razón es mejor hacer los pruebas antes.

Tipos de pruebas:

- Pruebas básicas
- Pruebas unitarias
 - Pruebas de integración ^{Ver si en otro funcionamiento} (conjunto de pruebas unitarias)
 - Pruebas de sistema ^{Verificamos y validación de req} ~~funcional~~ (probamos si una versión cumple con los req) ^{func y no func}
 - UAT, del lado del usuario/BO, pruebas para aceptar el producto
 - Pruebas de aceptación de usuario

TDD evolucionó a ATDD

Creación de TDD, ciclo Red-Green-Refactor

Avanzo escribiendo caso de prueba, eso va a fallar, ^{escribo código} ~~refactorizo~~ hasta que haga pasar el test (Verde). Luego Refactorizo

3 leyes TDD

- No escribir código antes de haber escrito un caso de prueba que falle.
- Con un test que falle ya es suficiente, no hay que escribir más tests.
- No vas a escribir más que lo suficiente para pasar el test.

Refactoring

Forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos.

Transformación del SW que preserva la estructura externa, y mejora la estructura interna. Se busca reducir el acoplamiento y aumentar la cohesión.

- Ciclo
1. La prueba de ~~esta~~ falla.
 2. Escribir el código interno para que la prueba pase.
 3. La prueba pasa.
 4. Aplicar Refactoring, se debe mejorar el código.

Precondiciones = setup = estado inicial necesario para ejecutar correctamente los casos de prueba